

Hosting and Publishing a Website with a Database on an AWS Linux Server

Internship Task — Deployment Documentation

Stack: PHP · MySQL · Apache (LAMP)

Server: AWS EC2 — Ubuntu Server 26.04 LTS

Prepared by: Faris K

Subject: Cloud Administration

Date: 28-07-2026

1. Task Description & Requirements

This document records the full process of hosting and publishing a website with a database on a cloud Linux server, as required by Internship Task 5. The exercise required demonstrating the ability to deploy a web application to cloud infrastructure and make it reliably accessible over the public internet, complete with a domain name and a valid SSL certificate.

Requirements Covered

- Source code of a website with a database, hosted on a cloud Linux server.
- An AWS account (Free Tier).
- A cloud server configured to meet the application's software dependencies.
- Full deployment: web server configuration, dependency installation, source code transfer.
- A domain name configured for the website.
- A valid SSL certificate via Certbot / Let's Encrypt, with HTTPS enforced.
- Thorough testing across browsers and devices.
- Documentation of every configuration, challenge, and solution.

Submission Summary

- Live URL: <https://faris.abrdns.com>
- Documentation: this document

2. Project Selection & Prerequisites

Rather than deploy an unstyled template, a personal portfolio website was hosted and then extended with a genuine database-backed feature: a contact form that writes every submission to a MySQL table. This keeps the deliverable simultaneously useful as (a) a valid submission for this task and (b) a live, presentable portfolio site.

Application Features

- A dark, responsive single-page portfolio (Projects, Skills, Experience, Contact sections) built with plain HTML/CSS.
- A working Contact form that submits to a PHP handler and inserts each message into a MySQL table.
- Every submission is stored with a name, email, message, and an automatic timestamp.
- HTTPS enforced site-wide via a free Let's Encrypt certificate.

Technology Stack

- Frontend: HTML5, CSS3 (custom, no framework).
- Backend: PHP 8.5 (mysqli extension).
- Database: MySQL 8.4, self-hosted on the same server.
- Infrastructure: AWS EC2 (Ubuntu Server 26.04 LTS), Apache2, Certbot / Let's Encrypt, CloudDNS free dynamic DNS.

Prerequisites

- An AWS account with Free Tier access.
- Basic familiarity with SSH and the Linux command line.
- A free CloudDNS account for a public hostname.

3. System Architecture

The diagram below summarizes the full request path: a user's browser resolves the public hostname through CloudDNS, reaches Apache over HTTPS, which serves the site directly and hands off the contact-form submission to PHP. PHP authenticates against a locally hosted, access-controlled MySQL instance and writes the message to the database.

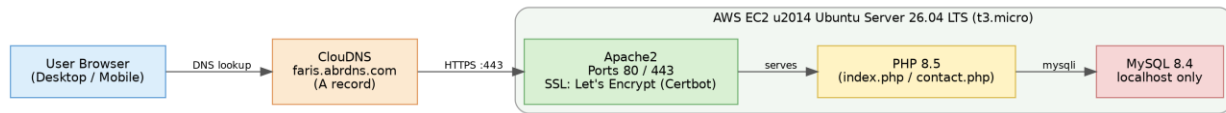


Figure 1: Deployment architecture on AWS EC2 (Ubuntu).

4. Task 1 — AWS Account & EC2 Instance Provisioning

An existing AWS account (Free Tier eligible) was used, with the EC2 instance provisioned in the Asia Pacific (Tokyo) region.

4.1 Launching the Instance

- AMI: Ubuntu Server 26.04 LTS (HVM), Free Tier eligible.
- Instance type: t3.micro.
- Key pair: new RSA key pair generated and downloaded (.pem).
- Security group inbound rules: SSH (22), HTTP (80), and HTTPS (443) opened.
- Storage: default 8 GiB volume.

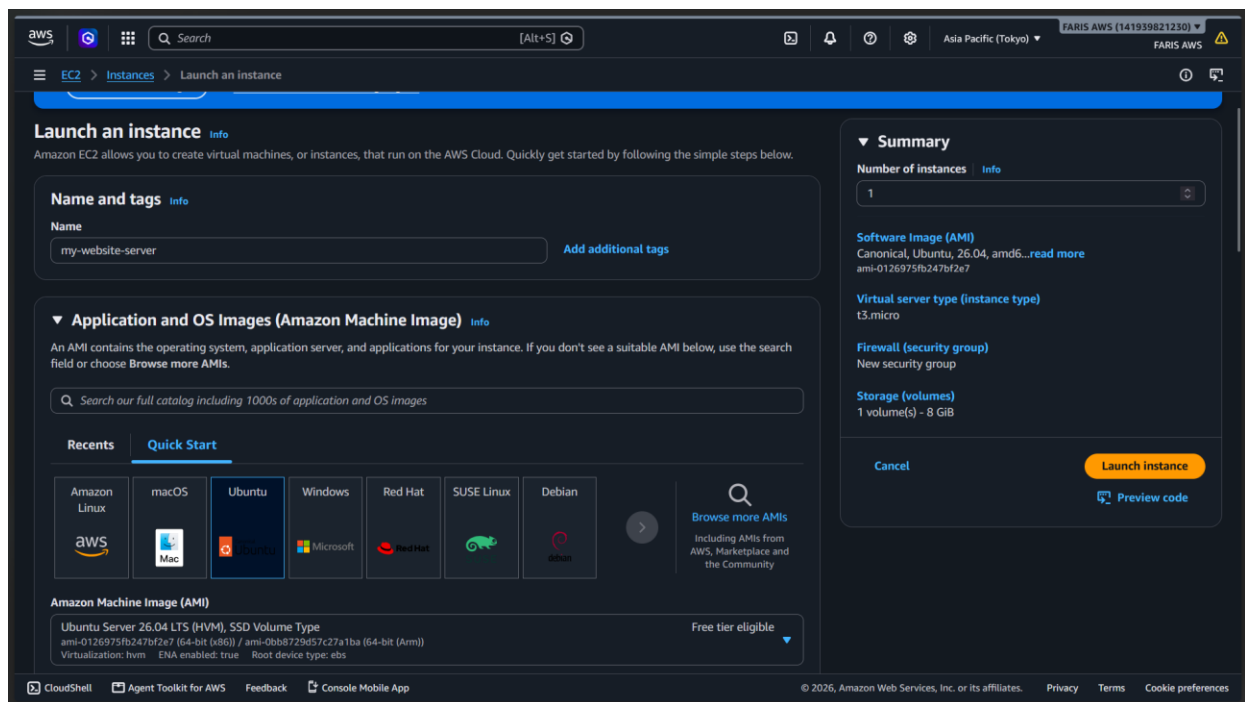


Figure 2: EC2 launch configuration — Ubuntu Server 26.04 LTS, t3.micro.

4.2 Security Group Configuration

Inbound rules were edited to allow SSH from a custom source, and HTTP/HTTPS from anywhere, so the site would be reachable by any visitor while SSH access remained restricted.

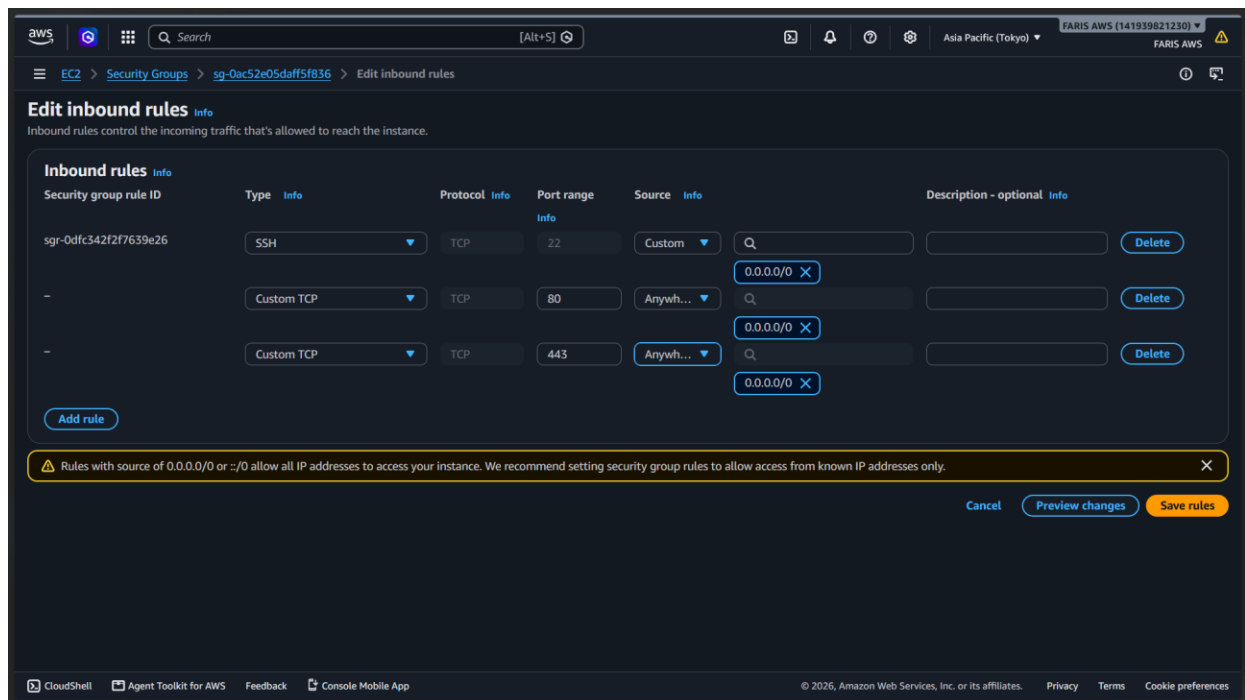


Figure 3: Security group inbound rules — ports 22, 80, and 443 opened.

5. Task 2 — Server Environment & Dependency Installation

This satisfies the requirement to configure the server so its software environment matches the application's needs. After connecting over SSH, the system packages were updated and the Apache, MySQL, and PHP components of a LAMP stack were installed:

```
sudo apt update && sudo apt upgrade -y
sudo apt install apache2 mysql-server php php-mysql libapache2-mod-php \
    php-cli php-curl php-xml unzip git -y
sudo systemctl enable apache2 mysql
sudo systemctl start apache2 mysql
```

```
ubuntu@ip-172-31-24-42: ~  
User sessions running outdated binaries:  
ubuntu @ session #1: sshd-session[1145,1262]  
ubuntu @ user manager: (sd-pam)[1152]  
  
No VM guests are running outdated hypervisor (qemu) binaries on this host.  
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.  
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2  
ubuntu@ip-172-31-24-42:~$ sudo systemctl status apache2  
● apache2.service - The Apache HTTP Server  
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)  
   Active: active (running) since Tue 2026-07-28 05:22:32 UTC; 9s ago  
  Invocation: 932d4b9db76c437fad7ddb8b75bdbc2d  
     Docs: https://httpd.apache.org/docs/2.4/  
   Main PID: 38572 (apache2)  
    Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:  0 B/sec"  
     Tasks: 6 (limit: 627)  
    Memory: 13.9M (peak: 20.5M)  
       CPU: 107ms  
   CGroup: /system.slice/apache2.service  
           └─38572 /usr/sbin/apache2 -k start -DFOREGROUND  
             └─38575 /usr/sbin/apache2 -k start -DFOREGROUND  
               └─38576 /usr/sbin/apache2 -k start -DFOREGROUND  
                 └─38577 /usr/sbin/apache2 -k start -DFOREGROUND  
                   └─38578 /usr/sbin/apache2 -k start -DFOREGROUND  
                     └─38580 /usr/sbin/apache2 -k start -DFOREGROUND  
  
Jul 28 05:22:32 ip-172-31-24-42 systemd[1]: Starting apache2.service - The Apache HTTP Server...  
Jul 28 05:22:32 ip-172-31-24-42 systemd[1]: Started apache2.service - The Apache HTTP Server.  
ubuntu@ip-172-31-24-42:~$
```

Figure 4: Apache2 installed, enabled, and running.

```
ubuntu@ip-172-31-24-42: ~  
User sessions running outdated binaries:  
ubuntu @ session #1: sshd-session[1145,1262]  
ubuntu @ user manager: (sd-pam)[1152]  
  
No VM guests are running outdated hypervisor (qemu) binaries on this host.  
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.  
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2  
ubuntu@ip-172-31-24-42:~$ sudo systemctl status apache2  
● apache2.service - The Apache HTTP Server  
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)  
   Active: active (running) since Tue 2026-07-28 05:22:32 UTC; 9s ago  
  Invocation: 932d4b9db76c437fad7ddb8b75bdbc2d  
     Docs: https://httpd.apache.org/docs/2.4/  
   Main PID: 38572 (apache2)  
    Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:  0 B/sec"  
     Tasks: 6 (limit: 627)  
    Memory: 13.9M (peak: 20.5M)  
       CPU: 107ms  
   CGroup: /system.slice/apache2.service  
           └─38572 /usr/sbin/apache2 -k start -DFOREGROUND  
             └─38575 /usr/sbin/apache2 -k start -DFOREGROUND  
               └─38576 /usr/sbin/apache2 -k start -DFOREGROUND  
                 └─38577 /usr/sbin/apache2 -k start -DFOREGROUND  
                   └─38578 /usr/sbin/apache2 -k start -DFOREGROUND  
                     └─38580 /usr/sbin/apache2 -k start -DFOREGROUND  
  
Jul 28 05:22:32 ip-172-31-24-42 systemd[1]: Starting apache2.service - The Apache HTTP Server...  
Jul 28 05:22:32 ip-172-31-24-42 systemd[1]: Started apache2.service - The Apache HTTP Server.  
ubuntu@ip-172-31-24-42:~$ sudo systemctl status mysql  
● mysql.service - MySQL Community Server  
   Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)  
   Active: active (running) since Tue 2026-07-28 05:22:18 UTC; 43s ago  
  Invocation: a4043da5a65a45a38ba741d0d20ec264  
     Main PID: 28680 (mysqld)  
    Status: "Server is operational"  
     Tasks: 35 (limit: 627)  
    Memory: 478.1M (peak: 486.3M)  
       CPU: 1.301s  
   CGroup: /system.slice/mysql.service  
           └─28680 /usr/sbin/mysqld  
  
Jul 28 05:22:16 ip-172-31-24-42 systemd[1]: Starting mysql.service - MySQL Community Server...  
Jul 28 05:22:18 ip-172-31-24-42 systemd[1]: Started mysql.service - MySQL Community Server.  
ubuntu@ip-172-31-24-42:~$
```

Figure 5: Apache2 and MySQL both active and running.

```
ubuntu@ip-172-31-24-42: ~$ sudo systemctl status apache2
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
* apache2.service - The Apache HTTP Server
  Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
  Active: active (running) since Tue 2026-07-28 05:22:32 UTC; 9s ago
  Invocation: 932d4b9db76c437fad7ddb8b75bdbc2d
  Docs: https://httpd.apache.org/docs/2.4/
  Main PID: 30572 (apache2)
  Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
  Tasks: 6 (limit: 627)
  Memory: 13.9M (peak: 20.5M)
  CPU: 107ms
  CGroup: /system.slice/apache2.service
          └─30572 /usr/sbin/apache2 -k start -DFOREGROUND
             └─30575 /usr/sbin/apache2 -k start -DFOREGROUND
                └─30576 /usr/sbin/apache2 -k start -DFOREGROUND
                   └─30577 /usr/sbin/apache2 -k start -DFOREGROUND
                      └─30578 /usr/sbin/apache2 -k start -DFOREGROUND
                         └─30580 /usr/sbin/apache2 -k start -DFOREGROUND

Jul 28 05:22:32 ip-172-31-24-42 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jul 28 05:22:32 ip-172-31-24-42 systemd[1]: Started apache2.service - The Apache HTTP Server.
ubuntu@ip-172-31-24-42:~$ sudo systemctl status mysql
* mysql.service - MySQL Community Server
  Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)
  Active: active (running) since Tue 2026-07-28 05:22:18 UTC; 43s ago
  Invocation: a4043da5a65a45a38ba741d0d20ec264
  Main PID: 28680 (mysqld)
  Status: "Server is operational"
  Tasks: 35 (limit: 627)
  Memory: 478.1M (peak: 486.3M)
  CPU: 1.301s
  CGroup: /system.slice/mysql.service
          └─28680 /usr/sbin/mysqld

Jul 28 05:22:16 ip-172-31-24-42 systemd[1]: Starting mysql.service - MySQL Community Server...
Jul 28 05:22:18 ip-172-31-24-42 systemd[1]: Started mysql.service - MySQL Community Server.
ubuntu@ip-172-31-24-42:~$ php -v
PHP 8.5.4 (cli) (built: Jul 16 2026 18:56:38) (NTS)
Copyright (c) The PHP Group
Built by Ubuntu
Zend Engine v4.5.4, Copyright (c) Zend Technologies
with Zend OPcache v8.5.4, Copyright (c), by Zend Technologies
ubuntu@ip-172-31-24-42:~$
```

Figure 6: PHP 8.5 installed and verified with `php -v`.

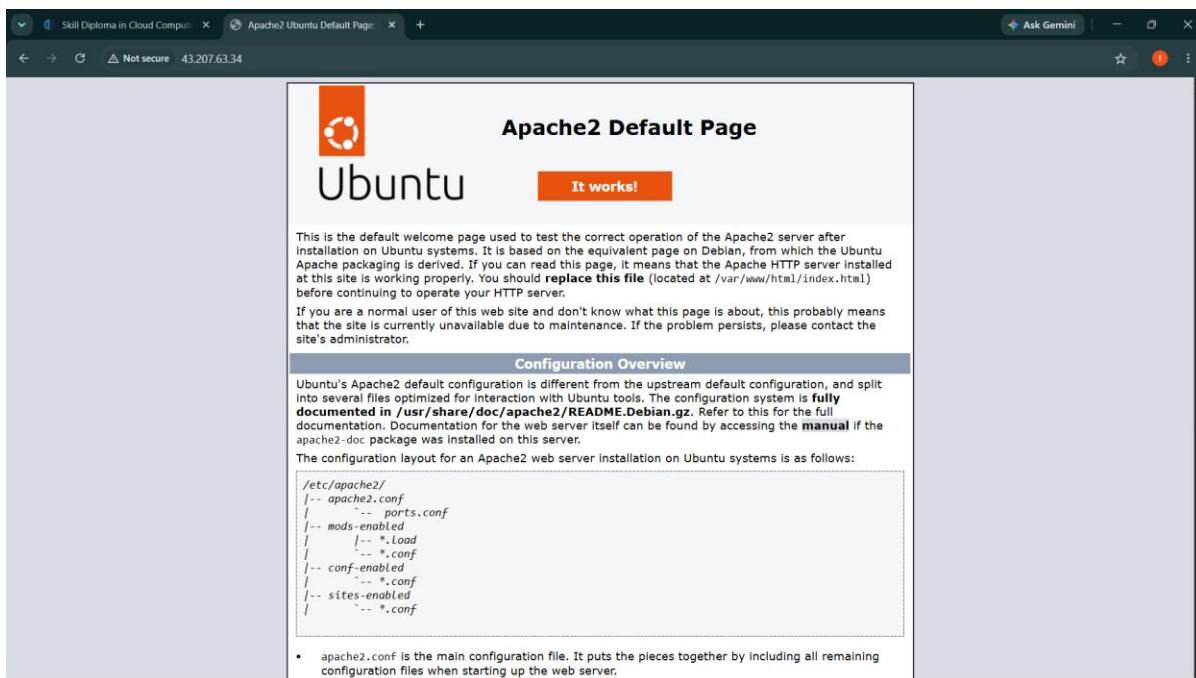


Figure 7: Apache default page confirmed reachable at the instance's public IP.

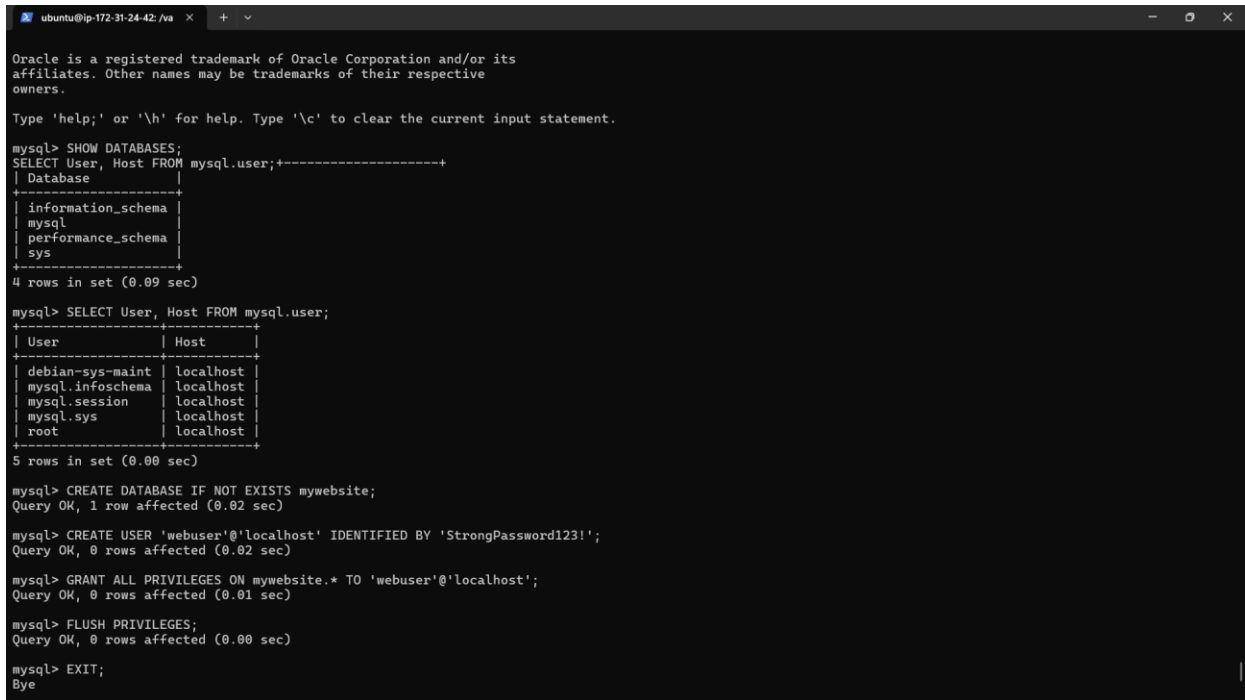
6. Task 3 — Database Configuration

MySQL was hardened with a dedicated, least-privilege application user rather than using the root account from the application layer:

```
sudo mysql -u root

CREATE DATABASE IF NOT EXISTS mywebsite;
CREATE USER 'webuser'@'localhost' IDENTIFIED BY '<strong-password>';
```

```
GRANT ALL PRIVILEGES ON mywebsite.* TO 'webuser'@'localhost';
FLUSH PRIVILEGES;
```



```
ubuntu@ip-172-31-24-42: /v$
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
SELECT User, Host FROM mysql.user;+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.09 sec)

mysql> SELECT User, Host FROM mysql.user;
+-----+-----+
| User | Host |
+-----+-----+
| debian-sys-maint | localhost |
| mysql.infoschema | localhost |
| mysql.session | localhost |
| mysql.sys | localhost |
| root | localhost |
+-----+-----+
5 rows in set (0.00 sec)

mysql> CREATE DATABASE IF NOT EXISTS mywebsite;
Query OK, 1 row affected (0.02 sec)

mysql> CREATE USER 'webuser'@'localhost' IDENTIFIED BY 'StrongPassword123!';
Query OK, 0 rows affected (0.02 sec)

mysql> GRANT ALL PRIVILEGES ON mywebsite.* TO 'webuser'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)

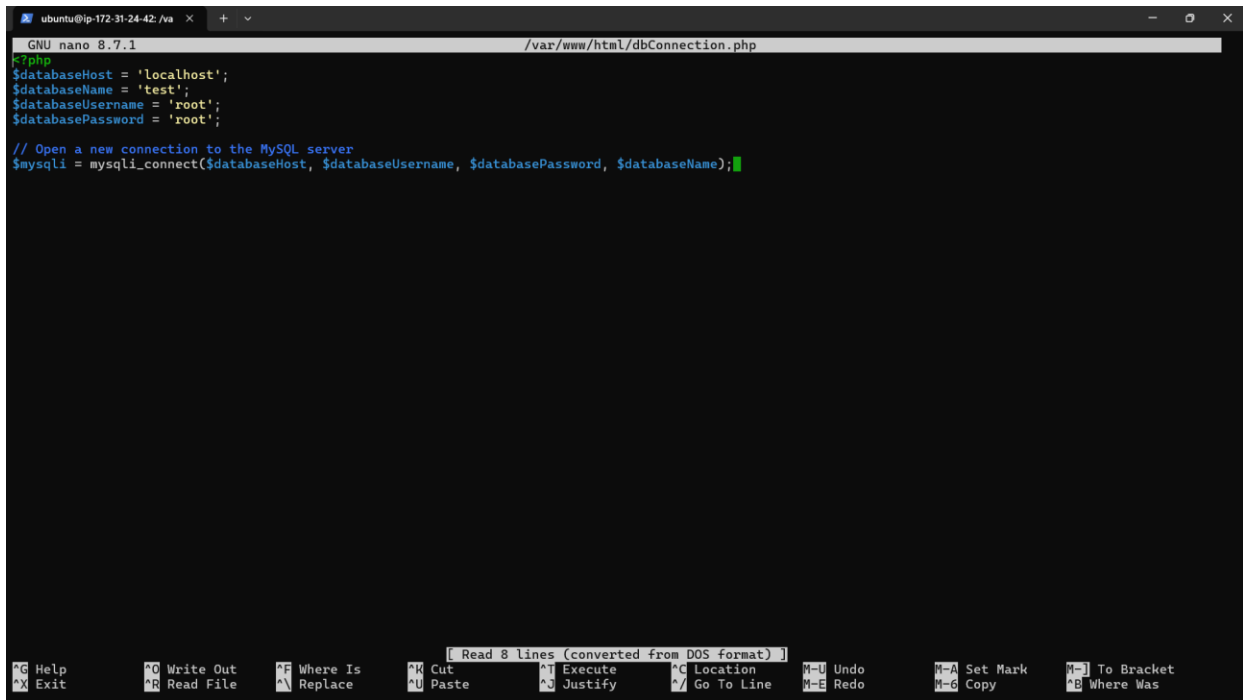
mysql> EXIT;
Bye
```

Figure 8: Dedicated database and least-privilege application user created successfully.

7. Task 4 — Application Deployment

7.1 Initial Attempt: Open-Source CRUD Boilerplate

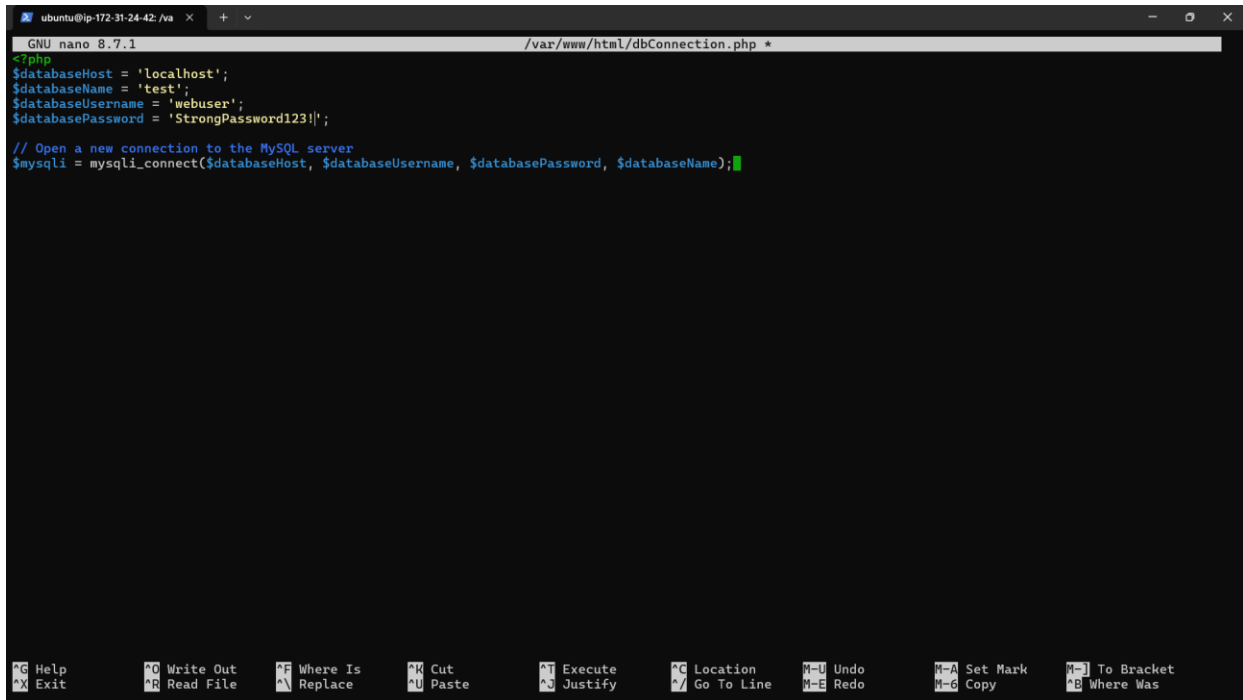
To validate the full PHP → MySQL data path quickly, an open-source PHP/MySQL CRUD boilerplate (chapagain/crud-php-simple) was cloned directly onto the server and its database.sql file imported. This confirmed end-to-end connectivity but, on review, its unstyled interface was not suitable as a portfolio-facing deliverable, so it was replaced in Section 7.2. Its database connection file is shown below, both before and after the credentials were corrected from the project's default placeholders to the real webuser account created in Task 3.



```
GNU nano 8.7.1 /var/www/html/dbConnection.php
<?php
$databaseHost = 'localhost';
$databaseName = 'test';
$databaseUsername = 'root';
$databasePassword = 'root';

// Open a new connection to the MySQL server
$mysqli = mysqli_connect($databaseHost, $databaseUsername, $databasePassword, $databaseName);
```

Figure 9: Default database connection file, using placeholder root credentials.



```
GNU nano 8.7.1 /var/www/html/dbConnection.php *
<?php
$databaseHost = 'localhost';
$databaseName = 'test';
$databaseUsername = 'webuser';
$databasePassword = 'StrongPassword123!';

// Open a new connection to the MySQL server
$mysqli = mysqli_connect($databaseHost, $databaseUsername, $databasePassword, $databaseName);
```

Figure 10: Database connection file updated with the real webuser credentials.

7.2 Final Deployment: Custom Portfolio Site

The unstyled boilerplate was removed and replaced with a custom-built, dark-themed portfolio site (index.php), keeping the working database connection but presenting it through an existing Contact form rather than a generic CRUD interface. A new handler, contact.php, was written to receive the form's POST data and insert it into a dedicated messages table:

```
<?php
$conn = new mysqli("localhost", "webuser", "<password>", "test");
```

```
$name    = $conn->real_escape_string($_POST['name']);  
$email    = $conn->real_escape_string($_POST['email']);  
$message = $conn->real_escape_string($_POST['message']);  
  
$sql = "INSERT INTO messages (name, email, message)  
        VALUES ('$name', '$email', '$message')";  
$conn->query($sql);
```

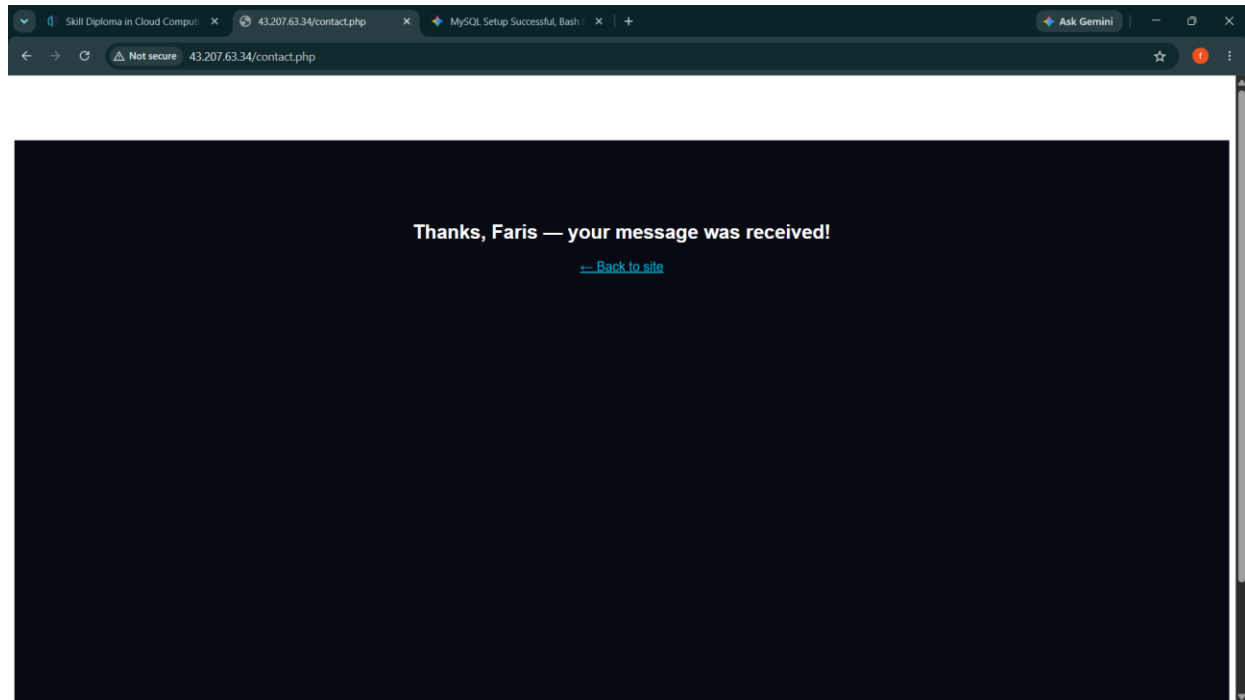


Figure 11: Contact form submission confirmed, with the record written to MySQL.

8. Task 5 — Domain Configuration

The task specifies Route 53 "or any other DNS service." Route 53 was considered, but it requires purchasing a domain outright or paying a recurring hosted-zone fee. Since this is a free-tier academic deployment, CloudDNS — a free DNS hosting provider — was used instead, which satisfies the same functional requirement: a stable public hostname for the SSL certificate to bind to.

A CloudDNS zone was configured with an A record pointing faris.abrdns.com (and www.faris.abrdns.com) directly at the EC2 instance's public IP address.

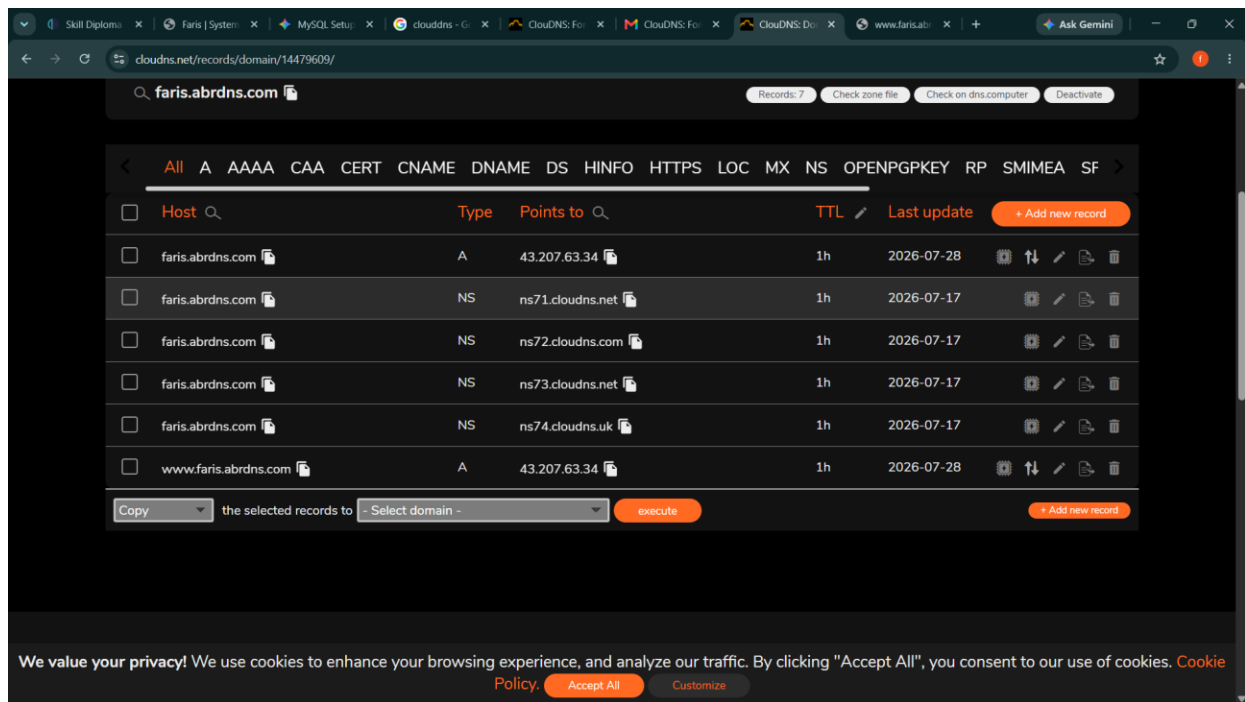


Figure 12: CloudDNS A records for faris.abrdns.com and www.faris.abrdns.com.

The site was confirmed reachable over the new hostname prior to enabling SSL:

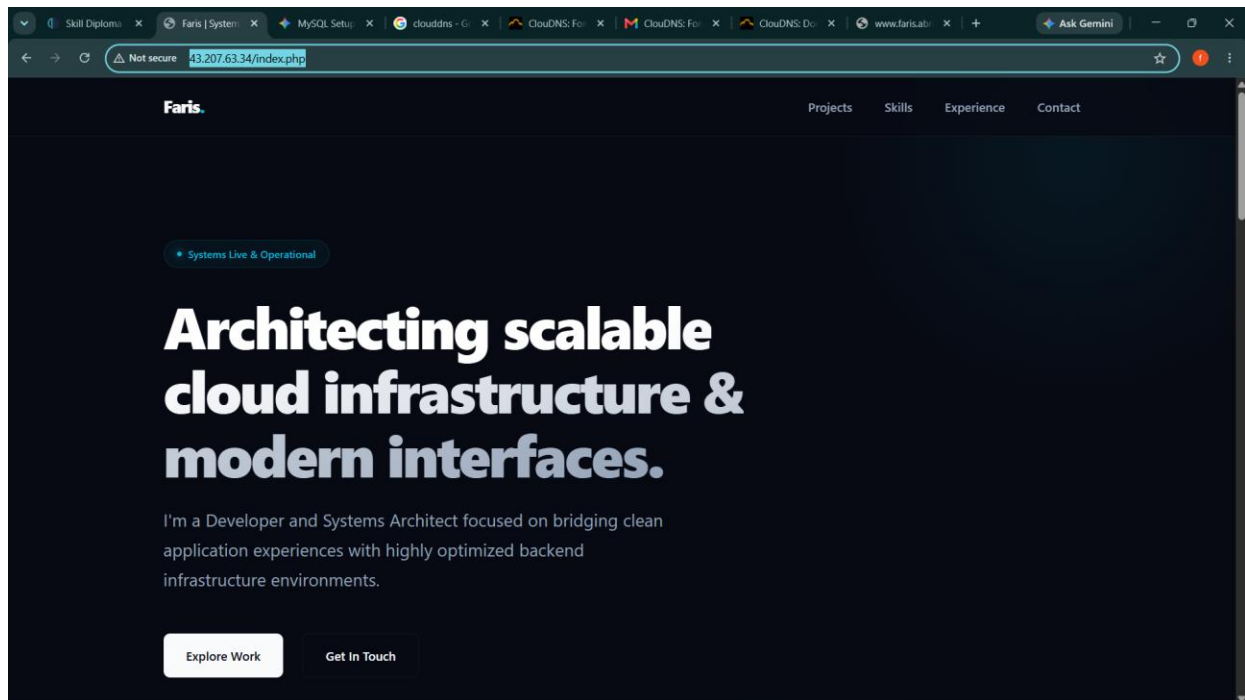


Figure 13: Portfolio site loading successfully over the domain, prior to HTTPS.

9. Task 6 — SSL Certificate via Certbot

Certbot and its Apache plugin were installed, then used to request and automatically deploy a free Let's Encrypt certificate for both the root domain and the www subdomain:

```
sudo apt install certbot python3-certbot-apache -y
```

```
sudo certbot --apache -d faris.abrdns.com -d www.faris.abrdns.com
```

```
ubuntu@ip-172-31-24-42: ~$ sudo certbot --apache -d faris.abrdns.com -d www.faris.abrdns.com
-----
Please read the Terms of Service at:
https://letsencrypt.org/documents/LE-SA-v1.8-July-06-2026.pdf
You must agree in order to register with the ACME server. Do you agree?
-----
(Y)es/(N)o: Y

-----
Would you be willing, once your first certificate is successfully issued, to
share your email address with the Electronic Frontier Foundation, a founding
partner of the Let's Encrypt project and the non-profit organization that
develops Certbot? We'd like to send you email about our work encrypting the web,
EFF news, campaigns, and ways to support digital freedom.
-----
(Y)es/(N)o: Y
Account registered.
Requesting a certificate for faris.abrdns.com and www.faris.abrdns.com

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/faris.abrdns.com/fullchain.pem
Key is saved at: /etc/letsencrypt/live/faris.abrdns.com/privkey.pem
This certificate expires on 2026-10-26.
These files will be updated when the certificate renews.
Certbot has set up a scheduled task to automatically renew this certificate in the background.

Deploying certificate
Successfully deployed certificate for faris.abrdns.com to /etc/apache2/sites-available/000-default-le-ssl.conf

We were unable to find a vhost with a ServerName or Address of www.faris.abrdns.com.
Which virtual host would you like to choose?
-----
1: 000-default.conf | | | Enabled
2: 000-default-le-ssl.conf | faris.abrdns.com | HTTPS | Enabled
-----
Select the appropriate number [1-2] then [enter] (press 'c' to cancel): 2
Successfully deployed certificate for www.faris.abrdns.com to /etc/apache2/sites-available/000-default-le-ssl.conf
Congratulations! You have successfully enabled HTTPS on https://faris.abrdns.com and https://www.faris.abrdns.com

-----
If you like Certbot, please consider supporting our work by:
* Donating to ISRG / Let's Encrypt: https://letsencrypt.org/donate
* Donating to EFF: https://eff.org/donate-le
-----
ubuntu@ip-172-31-24-42:~$
```

Figure 14: Certbot successfully issuing and deploying the SSL certificate — “Congratulations! You have successfully enabled HTTPS.”

Certbot automatically configured a scheduled renewal task, so the certificate refreshes in the background before its expiry date without manual intervention.

10. Task 7 — Testing & Verification

The deployed site was tested end-to-end across multiple desktop browsers and a mobile device to confirm accessibility, functional correctness, and responsive behaviour over HTTPS.

10.1 Desktop Browser Testing

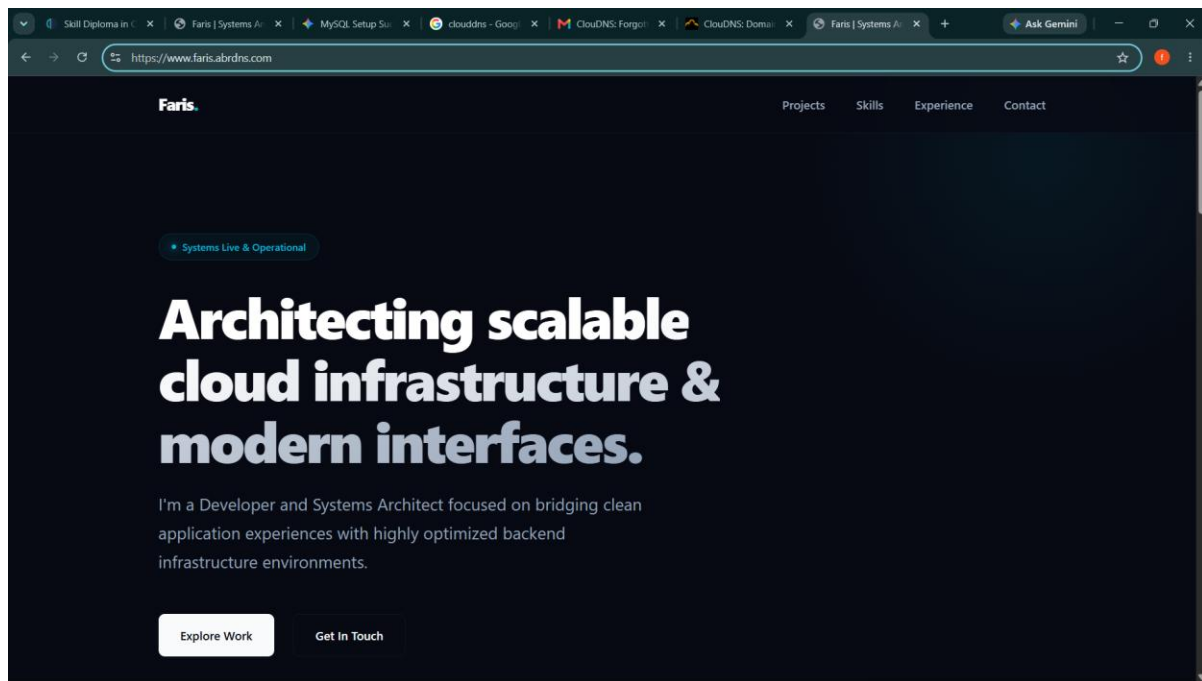


Figure 15: Site served over HTTPS in Chrome, with a valid padlock.

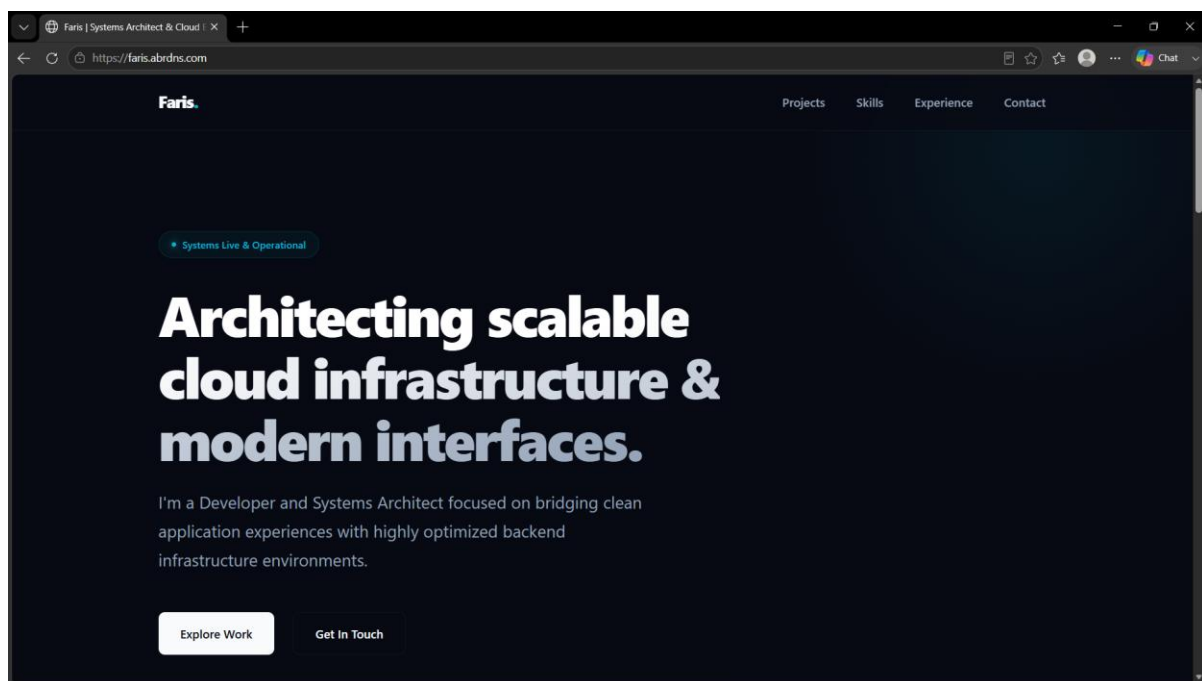


Figure 16: HTTPS padlock confirmed in a second browser (Microsoft Edge).

10.2 Mobile Device Testing

The responsive layout was verified on a mobile browser, confirming the site and its database-backed contact form both function correctly on a phone:



Figure 18: Portfolio site loading correctly on a mobile browser.

Additional Insights & Observations

- Validating the database connection first with a minimal, off-the-shelf CRUD example — before styling the final site — made it much faster to isolate whether a problem was in the PHP/MySQL wiring or in the application code itself.
- A LAMP stack (Apache, MySQL, PHP) remains a fast, low-overhead way to get a database-backed site running on a free-tier t3.micro instance, with no build step or process manager required.
- Free DNS services such as ClouDNS provide a practical, no-cost way to obtain a stable public hostname for a Let's Encrypt certificate, without needing to purchase a domain for an academic deployment.
- Certbot's Apache plugin automatically edits the live Apache configuration and sets up certificate auto-renewal, removing what would otherwise be the most error-prone manual step in enabling HTTPS.

- Testing on both desktop browsers and a mobile device early caught that the database write path worked identically regardless of platform, confirming the deployment (not just the front-end layout) was fully cross-device compatible.

Submission Summary

Live URL

<https://faris.abrdns.com>

Server Details

- Cloud provider: AWS (EC2, Free Tier, t3.micro)
- Operating system: Ubuntu Server 26.04 LTS
- Web server: Apache2
- Application runtime: PHP 8.5 (mysqli)
- Database: MySQL 8.4 (self-hosted, dedicated least-privilege user)
- Domain: ClouDNS free DNS — faris.abrdns.com
- SSL: Let's Encrypt via Certbot, auto-renewal enabled

Compliance

The portfolio site design and content were custom-built for this deployment. An open-source PHP CRUD boilerplate (chapagain/crud-php-simple) was used only during initial database-connectivity testing and is not part of the final live deployment.